Thinking in parallel

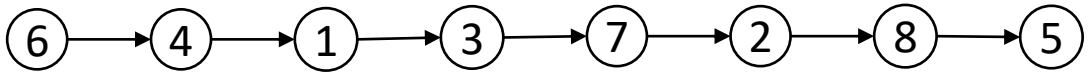
How to design algorithms for multi-processor systems

List-ranking problem

- A problem that arises in many context, such as computing tree functions and solving graph-theoretic problems
- It uses several of the basic techniques we introduced
- We will only see the first of the following two EREW PRAM algorithms
  - Time $O(\log n \log\log n)$ and work $O(n)$ (work optimal)
  - Time $O(\log n)$ and work $O(n)$ (work optimal)

List-ranking problem

LIST-RANKING PROBLEM
Input: An array S of n nodes representing a linked list, where node $S[i]$ is the successor of i in the list (we assume $S[i] = 0$ when i is the end of the list)
Output: An array R of n elements such that $R[i]$ contains the rank of i in the list, i.e., the hop-distance from i to the end of the list.



index	1	2	3	4	5	6	7	8
S	3	8	7	1	0	4	2	5

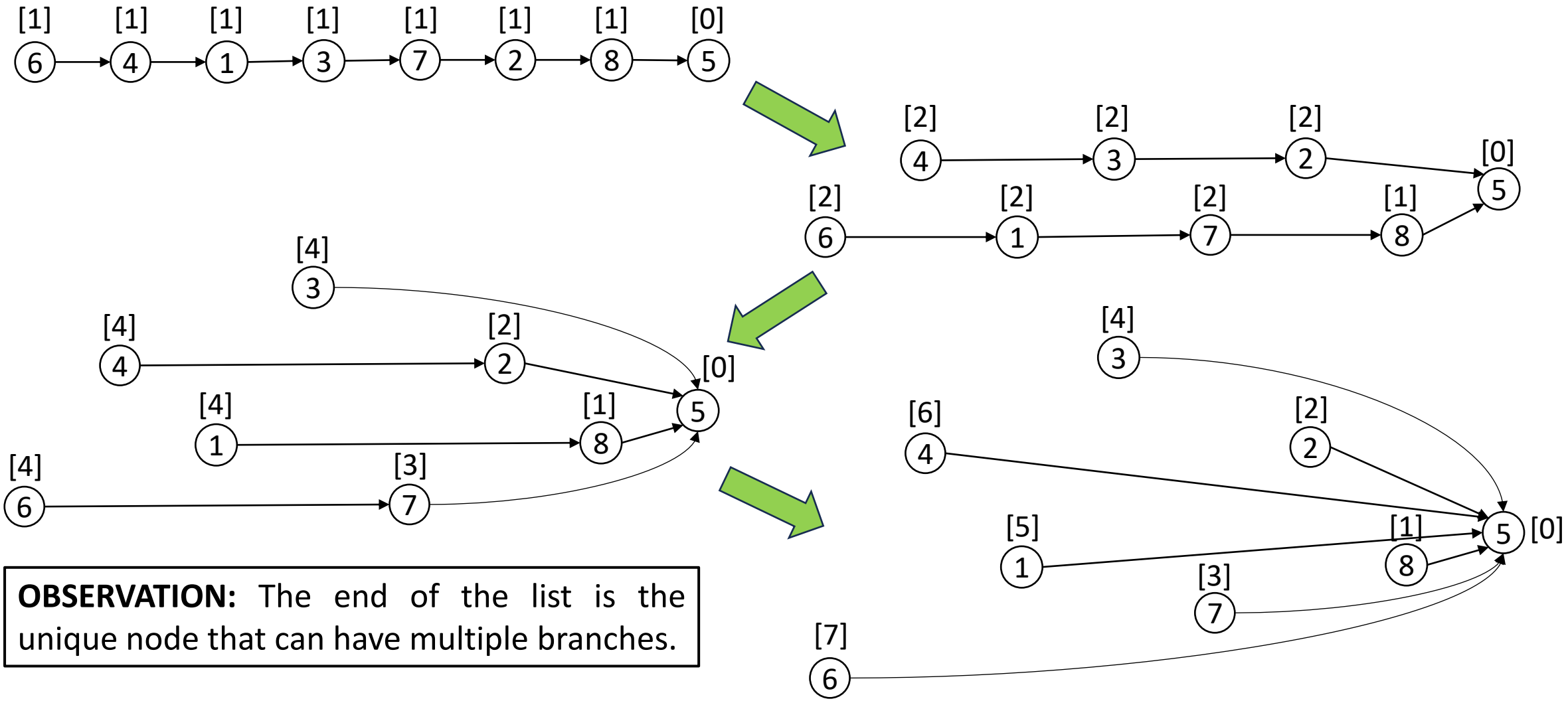
index	1	2	3	4	5	6	7	8
R	5	2	4	6	0	7	3	1

Easy to solve via a sequential $O(n)$-time algorithm

We already saw a technique for solving this problem

- Pointer Jumping (time $O(\log n)$ and work $O(n \log n)$)

An example with pointer jumping technique



OBSERVATION: The end of the list is the unique node that can have multiple branches.

Pointer-jumping based algorithm

ALGORITHM List_Ranking_with_Pointer_Jumping

```
1.  for 1 ≤ i ≤ n pardo
2.      if S[i] = 0 then R[i] ← 0
3.                  else  R[i] ← 1
4.  for 1 ≤ i ≤ n pardo
5.      Q[i] ← S[i]
6.      while(Q[i] ≠ 0 and Q[Q[i]] ≠ 0) do
7.          R[i] ← R[i] + R[Q[i]]
8.          Q[i] ← Q[Q[i]]
```

- Time $O(\log n)$
- Work $O(n \log n)$ (not work optimal)
- CREW PRAM algorithm
  - No concurrent write on same variable
  - Concurrent read on same variable in line 6: two nodes are pointing to the end of the list
- Can we avoid concurrent reads on same variable?
  - YES! First broadcast the end of the list!

Pointer-jumping based algorithm

ALGORITHM List_Ranking_with_Pointer_Jumping

```
for 1 ≤ i ≤ n pardo if S[i] = 0 then END[1] ← i   /* Detect the end of the list */
Broadcast END[1] to END[1..n]                     /* Broadcast it */
for 1 ≤ i ≤ n pardo
    if S[i] = 0 then R[i] ← 0
                else  R[i] ← 1
for 1 ≤ i ≤ n pardo
    Q[i] ← S[i]
    while(Q[i] ≠ END[i]) do                       /* Avoid concurrent reads */
        R[i] ← R[i] + R[Q[i]]
        Q[i] ← Q[Q[i]]
```

- Time $O(\log n)$
- Work $O(n \log n)$ (not work optimal)
- EREW PRAM algorithm

Pointer-jumping based algorithm

THEOREM: Given a linked list of n nodes, the List_Ranking_with_Pointer_Jumping algorithm computes the rank of each node i in time $O(\log n)$ and work $O(n \log n)$. Moreover, the algorithm can be implemented to run on the EREW PRAM.

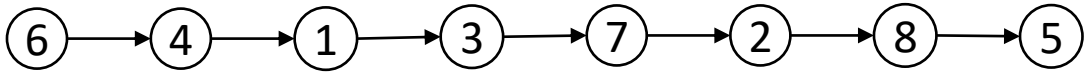
PROOF: Same as pointer jumping. The new part is to prove that the algorithm runs on the EREW PRAM.

Prove that, after each iteration, the end of the list is the only node with many incoming edges (i.e., more than one node points to the end of the list).

List-ranking problem: How to reduce the work?

- We have an EREW PRAM algorithm with time $O(\log n)$ and work $O(n \log n)$
- We aim at time $O(\log n \log\log n)$ and work $O(n)$ (work optimal)
- Potential approach
  1. Find a way to reduce the list to $O(n/\log n)$ nodes by removing most of the nodes
  2. Use the pointer-jumping based algorithm (time $O(\log n)$, work $O\left(\frac{n}{\log n}\log\frac{n}{\log n}\right) = O(n)$)
  3. Restore the original list and rank the nodes removed during step 1.
- Assume that we have the array P such that $P[i]$ is the predecessor of i in the list

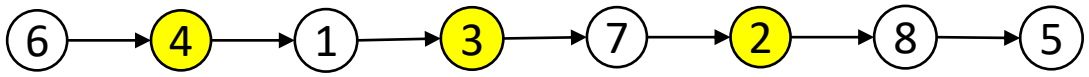
1. **for** $1 \le$ i $\le$ n **pardo**
2. P[i] $\leftarrow$ 0
3. **for** $1 \le$ i $\le$ n **pardo**
4. **if** S[i] $\neq$ 0 **then** P[S[i]] $\leftarrow$ i



index	1	2	3	4	5	6	7	8
S	3	8	7	1	0	4	2	5
P	4	7	1	6	8	0	3	2

Independent sets

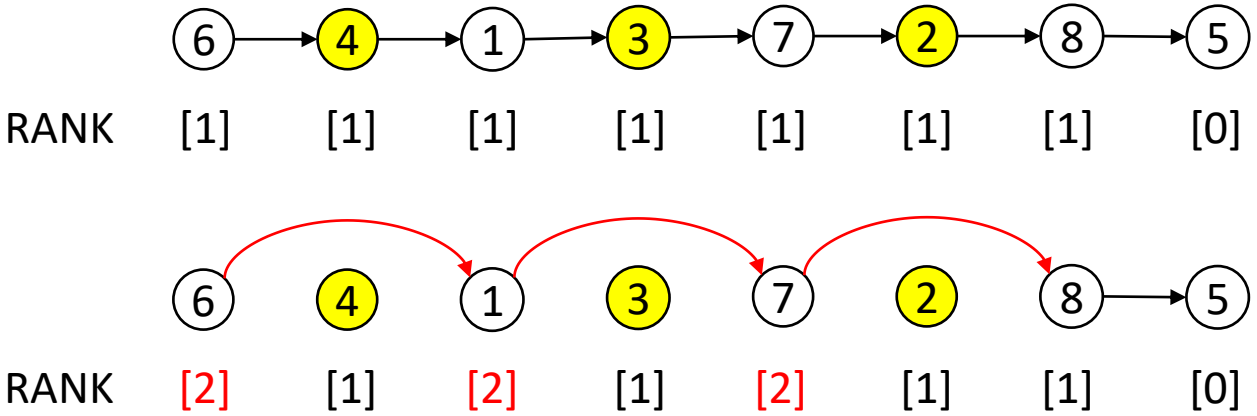
DEFINITION: A set I of nodes is an *independent set* if $S[i] \notin I$, for every $i \in I$.



$I = \{4,3,2\}$ is an independent set

- **IDEA:** Given an independent set I such that $P[i], S[i] \neq 0$ for every $i \in I$, we can remove I from the list concurrently in $O(1)$ time and $O(|I|)$ work

1. **for every** i ∈ I **pardo**
2. R[P[i]] ← R[P[i]]+R[i]
3. S[P[i]] ← S[i]
4. P[S[i]] ← P[i]



How to remove an independent set and keep track of it

ALGORITHM Remove_Independent_Set

/* We assume that P[i],S[i] $\neq$0 for every i $\in$ I */

/* U is an array that allows us to reconstruct the original linked list */

1. Assign consecutive numbers N[i], from 1 to |I|, to the elements of I, where $1 \leq i \leq |I|$
2. **for every i $\in$ I pardo**
3. U[N[i]] $\leftarrow$ (i, S[i], P[i])
4. R[P[i]] $\leftarrow$ R[i] + R[P[i]]
5. S[P[i]] $\leftarrow$ S[i]
6. P[S[i]] $\leftarrow$ P[i]

- Time $O(\log n)$
- Work $O(n)$ (work optimal)
- EREW PRAM algorithm

How to remove an independent set and keep track of it

LEMMA: Given a linked list of n nodes and an independent set I such that $P[i], S[i] \neq 0$ for every $i \in I$, the algorithm Remove_Independent_Set correctly removes the nodes of I from the list and updates the rank values of the leftover nodes in time $O(\log n)$ and work $O(n)$. Moreover, the algorithm can be implemented to run on the EREW PRAM.

PROOF: The correctness follows from the fact that no two nodes of I are adjacent.

For the time complexity bounds, all the steps, but step 1, take time $O(1)$ and work $O(n)$.

Step 1. can be implemented in time $O(\log n)$ and work $O(n)$ as follows:

a) Concurrently build the n-dimensional array N where $N[i] = 1$ if $i \in I$, and $N[i] = 0$ otherwise.
b) Compute prefix-sum of N.
c) For every $i \in I$, $N[i]$ is the number of i.

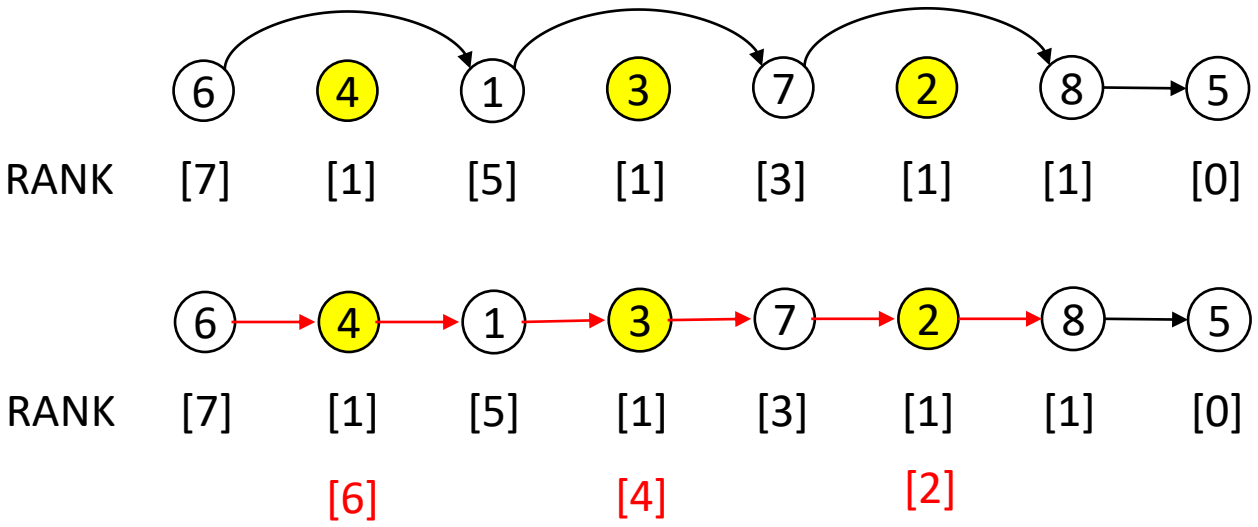
How to restore the linked list and rank the elements in I

- Assume that all elements of the shorter list have been ranked
- How do we restore the original list and rank the missing elements?

ALGORITHM Restore_List

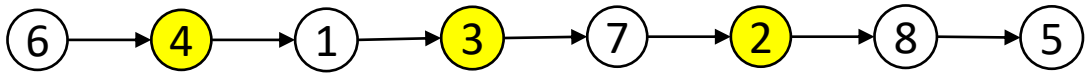
1. **for every** (i, S[i], P[i]) $\in$ U **pardo**
2. S[P[i]] $\leftarrow$ i
3. P[S[i]] $\leftarrow$ i
4. R[i] $\leftarrow$ R[S[i]] + R[i]

index	1	2	3
U	(2, 8, 7)	(3, 7, 1)	(4, 1, 6)

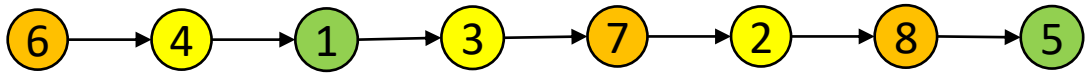


How to compute an independent set

- Can we reduce the problem to a problem we already know? What do we see?



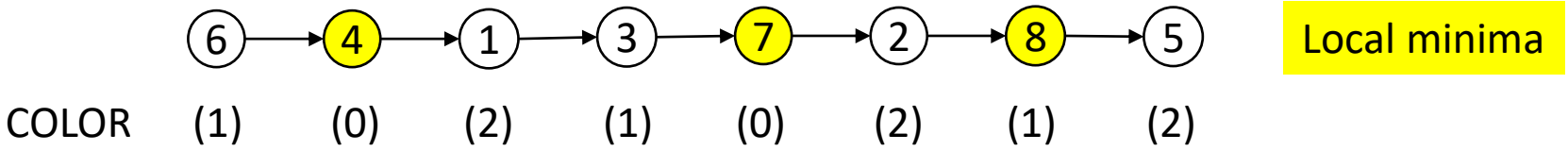
- What do we see now?



- Make the list circular and compute a 3-coloring ($O(\log n)$ time and $O(n)$ work in EREW)
- All the nodes of the same color form an independent set
- Choose a "good" independent set I
  - Large ($\Omega(n)$ size)
  - $P[i], S[i] \neq 0$ for every $i \in I$
- **ISSUE:** How can we detect a "good" independent set?

Local minimum

DEFINITION: A node i, with $P[i], S[i] \neq 0$, is a *local minimum* if the color of i is smaller than the color of $S[i]$ and the color of $P[i]$.



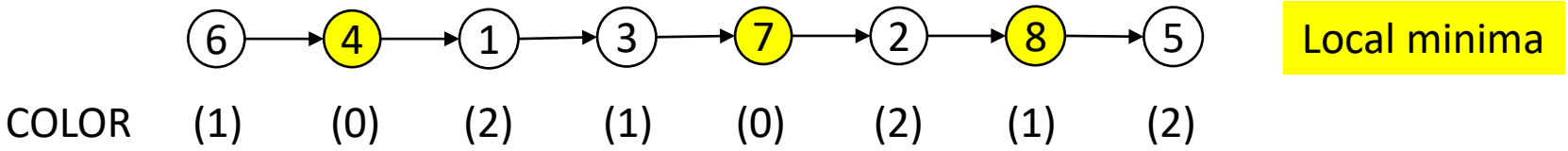
- Assume we have a 3-coloring
- What is the worst-case distance between two consecutive local minima?
  - the sequence of colors should be increasing and then decreasing, so at most 4

    Worst case sequence: $0, 1, 2, 1, 0$
- We have at least one local minima every 4 consecutive nodes

LEMMA: The set I of local minima is an independent set of size at least $\frac{n}{4}$.

Local minimum

LEMMA: The set I of local minima is an independent set of size at least $\frac{n}{4}$.



HOW TO COMPUTE LOCAL MINIMA

ALGORITHM Local_Minima // I is the Boolean array such that I[j]=1 iff j is a local minimum

1. **for $1 \leq j \leq n$ pardo**
2. if (COLOR[j]<COLOR[P[j]] and COLOR[j]<COLOR[S[j]]) then LM[j]=1 else LM[j]=0

- Time $O(1)$
- Work $O(n)$ (work optimal)
- EREW PRAM algorithm
  (first read COLOR[j], then read COLOR[P[j]] if P[j]$\neq 0$, finally read COLOR[S[i]] if $S[j] \neq 0$)

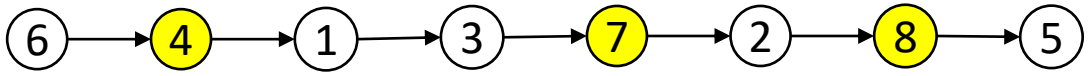
Putting all the pieces together

- **GOAL:** EREW PRAM algorithm with time $O(\log n \log\log n)$ and work $O(n)$
- WHAT WE HAVE (All the following algorithms are EREW):
  - 3-coloring: time $O(\log n)$ and work $O(n)$
  - Given a 3-coloring, we can find an independent set I of size at least $n/4$ in time $O(1)$ and work $O(n)$
  - We can remove I from the list in time $O(\log n)$ and work $O(n)$. The new list has size $\leq \frac{3}{4} \cdot n$
  - Once the elements of the new list are ranked, we can add I back and rank it in time $O(\log n)$ and work $O(n)$
  - The pointer-jumping based algorithm ranks a list of n nodes in time $O(\log n)$ and work $O(n \log n)$
- ANY IDEA ON HOW TO COMBINE EVERYTHING?
  - Divide and Conquer
    - **ISSUE:** if used alone, the analysis would give time $\Omega(\log^2 n)$ and work $O(n)$
  - Accelerated Cascading
    - Use Divide and Conquer until the list has size $O(n/\log n)$
    - Then use pointer-jumping based algorithm

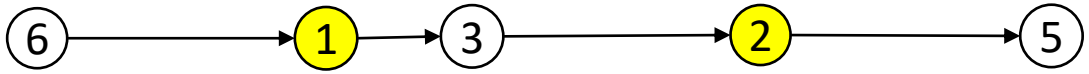
The work-optimal list-ranking algorithm: an example

DIVIDE AND CONQUER

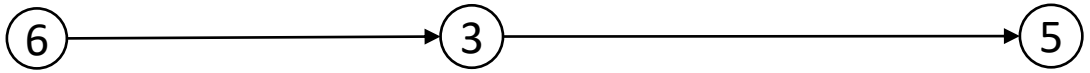
POINTER JUMPING



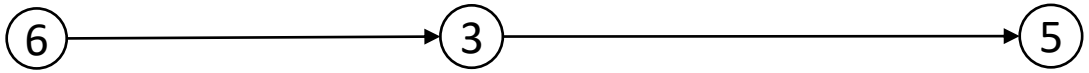
RANK	[1]	[1]	[1]	[1]	[1]	[1]	[1]	[0]
COLOR	(1)	(0)	(2)	(1)	(0)	(2)	(1)	(2)



RANK	[2]	[1]	[2]	[2]	[0]
COLOR	(2)	(1)	(2)	(0)	(2)



RANK	[3]	[4]	[0]

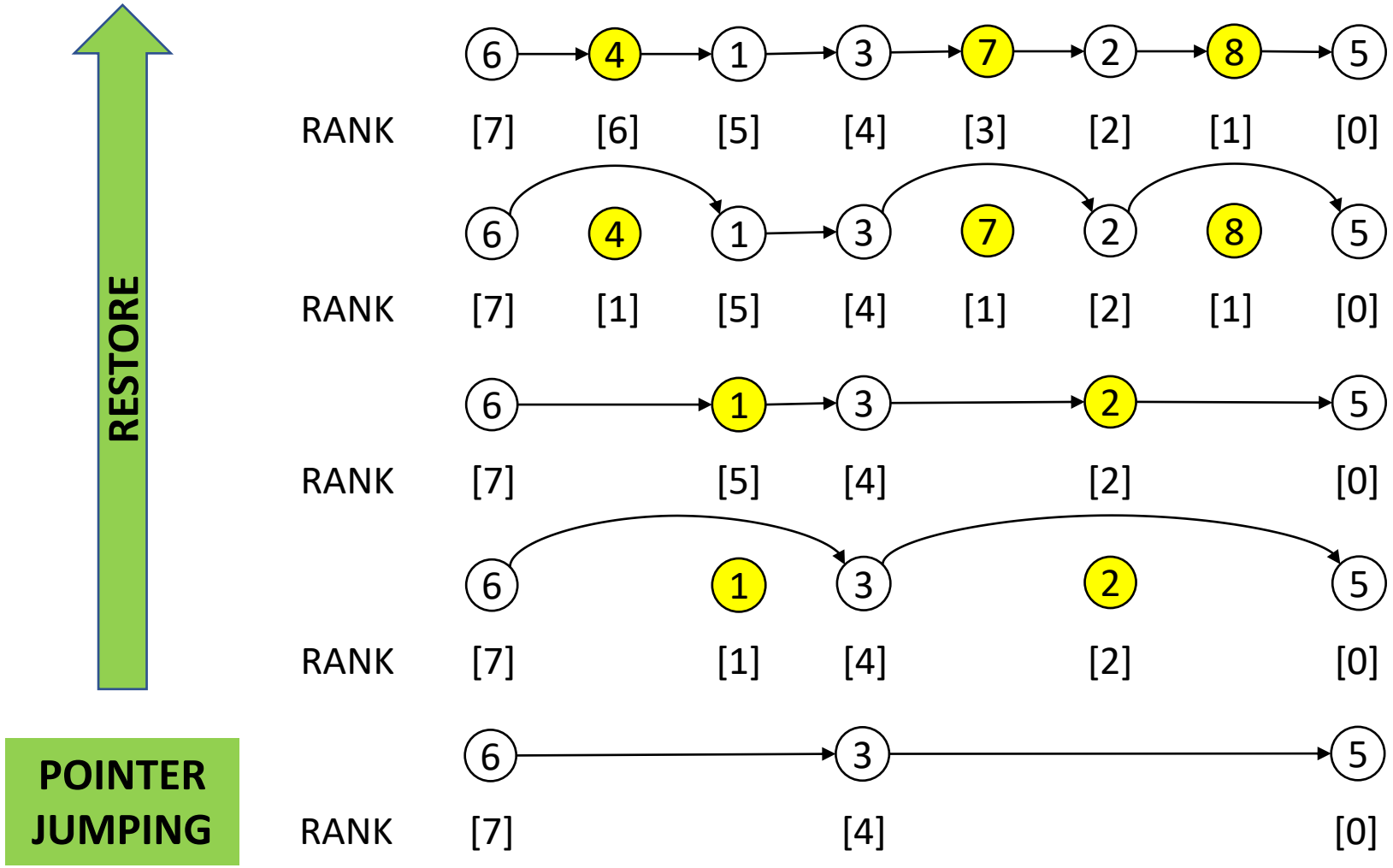


RANK	[7]	[4]	[0]

$n = 8, \mathrm{n}/\log_2 n \approx 3$

Local minima

The work-optimal list-ranking algorithm: an example



$n = 8, \mathrm{n}/\log_2 n \approx 3$

Local minima

An optimal list-ranking algorithm

ALGORITHM Optimal_List_Ranking

1. $n_0 \leftarrow n$; $k \leftarrow 0$
2. **while** $n_k > n/\log_2 n$ **do**
3. $k \leftarrow k + 1$
4. Color the list with 3 colors, and compute the independent set I of local minima
5. Run Remove_Independent_Set and keep track of I in U_K

   /* Next step is fundamental if we want to apply recursion on (3n)/4 instead of n */

   /* If m is the size of the list, we need to number the nodes of I in O(log m) time and O(m) work */

   /* without compacting the list, we would number the nodes of I in O(log n) time and O(n) work!!! */

6. Let n_k be the size of the remaining list. Compact the list into consecutive memory locations
7. Apply the pointer jumping technique to the resulting list
8. Restore the original list and rank all the removed nodes by reversing the process in the while loop **(BEWARE: Both algorithms Remove_Independent_Set and Restore_List need to be modified if we want to use them recursively!!!)**

An optimal list-ranking algorithm

THEOREM: Given a linked list of n nodes, the algorithm Optimal_List_Ranking correctly ranks all the elements of the list in in time $O(\log n \log\log n)$ and work $O(n)$. Moreover, the algorithm can be implemented to run on the EREW PRAM.

PROOF: The correctness proof follows from the correct implementation of steps 5 and 8 (exercise)

TIME AND WORK BOUNDS:
The k-th iteration of the while loop of algorithm Optimal_List_Ranking takes time $O(\log n)$ and work $O(n_k)$.

NUMBER OF ITERATIONS:

- prove by induction that, at the end of iteration k, the size of the new list is $n_k \leq \left(\frac{3}{4}\right)^k \cdot n$.
- $\Rightarrow$ the number of iterations required to reduce the size of the list to $O(n/\log n)$ is $O(\log\log n)$.

Hence, the time is $O(\log n \log\log n)$ and the work is $\sum_k O(n_k) = O\left(\sum_k n_k\right) = O\left(\sum_k \left(\frac{3}{4}\right)^k \cdot n\right) = O(n)$.

Check your understanding

Modify the algorithms Remove_Independent_Set and Restore_List to use them in steps 5 and 8 of algorithm Optimal_List_Ranking, respectively. Both algorithms should run in time $O(\log n)$ and work $O(n)$.

- Hint: Use an array Δ such that $\Delta[i]$ counts the number of nodes in the original linked list that, w.r.t. the new shorter list, are between i and $P[i]$.